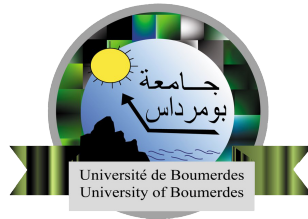


République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université de Boumerdès M'Hamed Bougara



Faculté des Sciences
Département d'Informatique

Spécialité : Intelligence Artificielle Appliquée (I2A)

Plateforme d'Optimisation des Emplois du Temps
d'Examens Universitaires

Lien de la plateforme :

<https://projetbdd.vercel.app/login>

Dépôt GitHub :

https://github.com/YousfiNacerEvo/projet_bdd

Vidéo de présentation :

<https://youtu.be/Hk-fmMcd2zw>

Préparé par :

MAHIDDINE Islem Amine
MESBAHI Mohammed Abedullah
YOUSFI Nacer
—Groupe 5—

Année universitaire 2025–2026

Résumé

Table des matières

Résumé	1
1 Introduction	2
1.1 Contexte	2
1.2 Problématique	2
1.3 Objectifs du projet	2
1.4 Périmètre du projet	2
2 Analyse des besoins et acteurs	3
2.1 Acteurs impliqués	3
2.2 Contraintes métier	3
2.3 Cas d'utilisation principaux	3
3 Spécification fonctionnelle	4
3.1 Fonctionnalités principales	4
3.2 Scénarios d'utilisation	4
4 Base de données	4
4.1 Introduction à la base de données	4
5 Méthodologie et algorithmes	9
5.1 Approche choisie	9
5.2 Justification	9
5.3 Algorithme d'optimisation	9
6 Architecture technique	11
6.1 Choix des technologies	11
6.2 Structure générale du projet	12
6.3 Structure du code	13
7 Déploiement et utilisation	13
7.1 Choix des plateformes de déploiement	13
7.2 Processus d'utilisation	13
8 Conclusion	14
Remarque	14

1 Introduction

1.1 Contexte

La planification des examens est une tâche complexe et critique pour nos établissements. Elle implique la coordination des étudiants, enseignants, salles, horaires et contraintes propres à chaque département. Dans les grandes universités, cette tâche devient rapidement difficile à gérer manuellement en raison du volume élevé de données, ce qui engendre un risque important d'erreurs.

L'élaboration des emplois du temps repose sur des procédures semi-manuelles, souvent longues et sujettes à des conflits, comme une mauvaise utilisation des ressources ou le chevauchement d'examens pour un même étudiant. Ces problèmes entraînent une surcharge administrative et affectent la qualité de l'organisation.

1.2 Problématique

L'organisation des examens doit respecter un ensemble de contraintes strictes et institutionnelles, telles qu'un étudiant n'ayant pas plus d'un examen par jour, la distribution équitable entre enseignants ou la priorité donnée à certains départements.

Face à cette complexité, les méthodes classiques sont insuffisantes et peu évolutives. Il est alors nécessaire de disposer d'un système automatisé capable de générer des emplois du temps optimisés, cohérents et adaptables aux changements, tout en réduisant le nombre de conflits et le temps de la tâche.

1.3 Objectifs du projet

L'objectif principal de ce projet est de concevoir et de développer une plateforme permettant d'optimiser les emplois du temps d'examens universitaires. Cette plateforme a pour but de :

- modéliser les données académiques nécessaires à la planification des emplois ;
- formuler le problème comme un problème d'optimisation avec contraintes ;
- générer des emplois du temps qui respectent les règles pédagogiques et logistiques ;
- offrir une interface permettant aux responsables de consulter et valider les résultats.

1.4 Périmètre du projet

Le projet se concentre sur la planification des examens au sein d'une université, en considérant les étudiants, les modules, les enseignants, les salles et les créneaux horaires disponibles. Les données utilisées sont générées de manière synthétique afin de simuler des scénarios à grande échelle.

Certaines fonctionnalités avancées, telles que la gestion des imprévus, ne sont pas prises en compte dans ce projet. Le système vise à démontrer l'efficacité d'une approche d'optimisation automatique dans un contexte académique réaliste.

2 Analyse des besoins et acteurs

2.1 Acteurs impliqués

Le système de planification des examens implique plusieurs acteurs ayant des rôles distincts. Chaque acteur interagit avec la plateforme selon ses responsabilités spécifiques.

- **Étudiant** : consulte son emploi du temps d'examens, les salles et les horaires, et vérifie l'absence de conflits.
- **Enseignant** : consulte les examens à surveiller ainsi que sa charge de surveillance.
- **Doyen** : supervise l'organisation globale des examens et valide la version finale de l'emploi du temps.
- **Administrateur des examens** : gère les données académiques et lance le processus de génération des emplois du temps.
- **Chef de département** : vérifie la cohérence des emplois du temps de son département et propose des ajustements si nécessaire.

2.2 Contraintes métier

Le système doit respecter un ensemble de contraintes académiques et organisationnelles afin de garantir la validité des emplois du temps générés.

- Un étudiant ne peut pas passer deux examens au même créneau horaire.
- Un étudiant ne peut pas avoir plus d'un examen par jour.
- La capacité maximale des salles doit être respectée.
- Un enseignant ne peut pas surveiller deux examens simultanément.
- La charge de surveillance doit être répartie équitablement entre les enseignants.
- Certains examens peuvent être prioritaires selon les départements.

2.3 Cas d'utilisation principaux

Les cas d'utilisation suivants décrivent les principales interactions entre les acteurs et la plateforme de planification des examens.

- **Gestion des données académiques** : l'administrateur des examens importe, modifie et supprime les informations relatives aux étudiants, enseignants, modules, salles et créneaux horaires.
- **Génération de l'emploi du temps** : l'administrateur lance le processus d'optimisation afin de produire automatiquement un emploi du temps respectant les contraintes définies.
- **Consultation des emplois du temps** : les étudiants et les enseignants consultent leurs plannings respectifs via la plateforme.
- **Validation départementale** : le chef de département analyse l'emploi du temps de son département et signale d'éventuelles incohérences.
- **Validation finale** : le doyen valide officiellement l'emploi du temps avant sa diffusion.

3 Spécification fonctionnelle

3.1 Fonctionnalités principales

La plateforme d’optimisation des emplois du temps d’examens offre les fonctionnalités suivantes :

- **Authentification des utilisateurs** : chaque acteur accède à la plateforme via un système de connexion sécurisé selon son rôle.
- **Gestion des données académiques** : ajout, modification et suppression des étudiants, enseignants, modules, salles et créneaux horaires.
- **Génération automatique du planning** : création d’un emploi du temps optimisé en respectant les contraintes définies.
- **Consultation du planning** : affichage des emplois du temps par étudiant, enseignant, salle ou département.
- **Validation du planning** : approbation du planning par le chef de département puis par le doyen.
- **Rejet et correction du planning** : possibilité de rejeter un planning avec retour vers l’administrateur pour correction.

3.2 Scénarios d’utilisation

Scénario 1 : Génération du planning

L’administrateur des examens se connecte à la plateforme, importe les données académiques et lance le processus de génération du planning. Le système produit automatiquement un emploi du temps optimisé.

Scénario 2 : Validation départementale

Le chef de département consulte le planning de son département. Il peut soit valider le planning, soit le rejeter en signalant les anomalies.

Scénario 3 : Validation finale

Le doyen consulte le planning validé par les chefs de département et procède à la validation finale avant diffusion.

Scénario 4 : Consultation

Les étudiants et enseignants consultent leurs emplois du temps respectifs via leur interface personnelle.

Scénario 5 : Rejet et correction

En cas de rejet, l’administrateur modifie les paramètres ou les données, puis relance la génération du planning.

4 Base de données

4.1 Introduction à la base de données

La base de données constitue le cœur du système de planification des examens. Elle permet de stocker, organiser et gérer l’ensemble des informations académiques nécessaires, telles que les étudiants, enseignants, modules, formations, salles, périodes et créneaux horaires.

L'objectif principal de cette base est de garantir :

- **Intégrité des données** : assurer la cohérence et la validité des informations, par exemple en empêchant la création d'inscriptions pour des modules inexistants.
- **Accessibilité** : permettre un accès rapide et fiable aux données pour l'ensemble des fonctionnalités de la plateforme.
- **Extensibilité** : faciliter l'ajout futur de nouvelles entités ou de nouvelles relations, telles que des contraintes supplémentaires ou des types d'examens spécifiques.
- **Optimisation des requêtes** : permettre une génération rapide des emplois du temps et la vérification des contraintes grâce à une structuration efficace des tables et des relations.

Cette section présente les principales tables de la base de données et leurs relations, ainsi que les contraintes mises en place pour assurer le bon fonctionnement de la plateforme.

Table periode_examen

```
1 CREATE TABLE public.periode_examen (  
2   id_periode integer NOT NULL DEFAULT nextval('periode_examen_id_periode_seq'::  
   regclass),  
3   nom character varying NOT NULL,  
4   date_debut date NOT NULL,  
5   date_fin date NOT NULL,  
6   type character varying CHECK (type IN ('session1','session2','rattrapage')),  
7   CONSTRAINT periode_examen_pkey PRIMARY KEY (id_periode)  
8 );
```

Table salle

```
1 CREATE TABLE public.salle (  
2   id_salle integer NOT NULL DEFAULT nextval('salle_id_salle_seq'::regclass),  
3   nom character varying NOT NULL,  
4   capacite integer CHECK (capacite > 0),  
5   type character varying CHECK (type IN ('salle','amphi')),  
6   batiment character varying NOT NULL,  
7   capacite_normale integer,  
8   capacite_examen integer,  
9   CONSTRAINT salle_pkey PRIMARY KEY (id_salle)  
10 );
```

Table equipement

```
1 CREATE TABLE public.equipement (  
2   id_equipement integer NOT NULL DEFAULT nextval('equipement_id_equipement_seq'  
   ::regclass),  
3   nom character varying NOT NULL,  
4   description text,  
5   CONSTRAINT equipement_pkey PRIMARY KEY (id_equipement)  
6 );
```

Table salle_equipement

```
1 CREATE TABLE public.salle_equipement (  
2   id_salle integer NOT NULL,  
3   id_equipement integer NOT NULL,  
4   quantite integer CHECK (quantite > 0),  
5   CONSTRAINT salle_equipement_pkey PRIMARY KEY (id_salle, id_equipement),  
6   CONSTRAINT salle_equipement_id_salle_fkey FOREIGN KEY (id_salle) REFERENCES  
   public.salle(id_salle),  
7   CONSTRAINT salle_equipement_id_equipement_fkey FOREIGN KEY (id_equipement)  
   REFERENCES public.equipement(id_equipement)  
8 );
```

Table professeur

```
1 CREATE TABLE public.professeur (  
2   id_prof integer NOT NULL DEFAULT nextval('professeur_id_prof_seq'::regclass),  
3   nom character varying NOT NULL,  
4   prenom character varying NOT NULL,  
5   date_naissance date NOT NULL CHECK (date_naissance < CURRENT_DATE),  
6   email character varying NOT NULL UNIQUE,  
7   grade character varying,  
8   id_dept integer NOT NULL,  
9   nb_surveillances_periode integer DEFAULT 0,  
10  CONSTRAINT professeur_pkey PRIMARY KEY (id_prof),  
11  CONSTRAINT professeur_id_dept_fkey FOREIGN KEY (id_dept) REFERENCES public.  
   departement(id_dept)  
12 );
```

Table inscription

```
1 CREATE TABLE public.inscription (  
2   id_etudiant integer NOT NULL,  
3   id_module integer NOT NULL,  
4   note numeric,  
5   CONSTRAINT inscription_pkey PRIMARY KEY (id_etudiant, id_module),  
6   CONSTRAINT inscription_id_etudiant_fkey FOREIGN KEY (id_etudiant) REFERENCES  
   public.etudiant(id_etudiant),  
7   CONSTRAINT inscription_id_module_fkey FOREIGN KEY (id_module) REFERENCES  
   public.module(id_module)  
8 );
```

Table session_examen

```
1 CREATE TABLE public.session_examen (  
2   id_session integer NOT NULL DEFAULT nextval('session_examen_id_session_seq'::  
   regclass),  
3   id_examen integer NOT NULL,  
4   id_salle integer NOT NULL,
```

```

5  nb_places_allouees integer CHECK (nb_places_allouees > 0),
6  CONSTRAINT session_examen_pkey PRIMARY KEY (id_session),
7  CONSTRAINT session_examen_id_examen_fkey FOREIGN KEY (id_examen) REFERENCES
    public.examen(id_examen),
8  CONSTRAINT session_examen_id_salle_fkey FOREIGN KEY (id_salle) REFERENCES
    public.salle(id_salle)
9 );

```

Table surveillance

```

1  CREATE TABLE public.surveillance (
2  id_examen integer NOT NULL,
3  id_prof integer NOT NULL,
4  role character varying DEFAULT 'surveillant' CHECK (role IN ('responsable', '
    surveillant')),
5  CONSTRAINT surveillance_pkey PRIMARY KEY (id_examen, id_prof),
6  CONSTRAINT surveillance_id_examen_fkey FOREIGN KEY (id_examen) REFERENCES
    public.examen(id_examen),
7  CONSTRAINT surveillance_id_prof_fkey FOREIGN KEY (id_prof) REFERENCES public.
    professeur(id_prof)
8 );

```

Table planning_runs

```

1  CREATE TABLE public.planning_runs (
2  id uuid NOT NULL DEFAULT gen_random_uuid(),
3  created_by uuid,
4  scope text NOT NULL CHECK (scope IN ('global', 'departement', 'formation')),
5  dept_id bigint,
6  formation_id bigint,
7  status text NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'running', '
    done', 'failed')),
8  published boolean DEFAULT false,
9  started_at timestamp with time zone,
10 ended_at timestamp with time zone,
11 metrics jsonb DEFAULT '{}'::jsonb,
12 created_at timestamp with time zone DEFAULT now(),
13 status_admin text DEFAULT 'draft' CHECK (status_admin IN ('draft', 'submitted')
    ),
14 status_doyen text DEFAULT 'pending' CHECK (status_doyen IN ('pending', '
    approved', 'rejected')),
15 submitted_at timestamp with time zone,
16 approved_at timestamp with time zone,
17 approved_by uuid,
18 rejected_at timestamp with time zone,
19 rejected_by uuid,
20 rejection_reason text,
21 published_at timestamp with time zone,
22 CONSTRAINT planning_runs_pkey PRIMARY KEY (id),
23 CONSTRAINT planning_runs_created_by_fkey FOREIGN KEY (created_by) REFERENCES
    auth.users(id),

```

```

24  CONSTRAINT planning_runs_approved_by_fkey FOREIGN KEY (approved_by) REFERENCES
    auth.users(id),
25  CONSTRAINT planning_runs_rejected_by_fkey FOREIGN KEY (rejected_by) REFERENCES
    auth.users(id)
26 );

```

Table planning_items

```

1  CREATE TABLE public.planning_items (
2    id uuid NOT NULL DEFAULT gen_random_uuid(),
3    run_id uuid,
4    module_id bigint,
5    salle_id bigint,
6    creneau_id bigint,
7    expected_students integer,
8    notes text,
9    surveillants jsonb DEFAULT '[]'::jsonb,
10   CONSTRAINT planning_items_pkey PRIMARY KEY (id),
11   CONSTRAINT planning_items_salle_id_fkey FOREIGN KEY (salle_id) REFERENCES
    public.salle(id_salle),
12   CONSTRAINT planning_items_run_id_fkey FOREIGN KEY (run_id) REFERENCES public.
    planning_runs(id),
13   CONSTRAINT planning_items_module_id_fkey FOREIGN KEY (module_id) REFERENCES
    public.module(id_module),
14   CONSTRAINT planning_items_creneau_id_fkey FOREIGN KEY (creneau_id) REFERENCES
    public.creneau(id_creneau)
15 );

```

Table users_meta

```

1  CREATE TABLE public.users_meta (
2    id uuid NOT NULL,
3    role text NOT NULL CHECK (role IN ('admin_examens', 'chef_dept', 'doyen', 'prof',
    'etudiant')),
4    dept_id bigint,
5    formation_id bigint,
6    created_at timestamp with time zone NOT NULL DEFAULT now(),
7    updated_at timestamp with time zone NOT NULL DEFAULT now(),
8    id_prof integer,
9    id_etudiant integer,
10   CONSTRAINT users_meta_pkey PRIMARY KEY (id),
11   CONSTRAINT users_meta_id_fkey FOREIGN KEY (id) REFERENCES auth.users(id),
12   CONSTRAINT users_meta_dept_id_fkey FOREIGN KEY (dept_id) REFERENCES public.
    departement(id_dept),
13   CONSTRAINT users_meta_formation_id_fkey FOREIGN KEY (formation_id) REFERENCES
    public.formation(id_formation),
14   CONSTRAINT users_meta_id_prof_fkey FOREIGN KEY (id_prof) REFERENCES public.
    professeur(id_prof),
15   CONSTRAINT users_meta_id_etudiant_fkey FOREIGN KEY (id_etudiant) REFERENCES
    public.etudiant(id_etudiant)

```

5 Méthodologie et algorithmes

5.1 Approche choisie

La génération automatique des emplois du temps d'examens universitaires constitue un problème d'optimisation combinatoire sous contraintes multiples. Ce type de problème est reconnu comme étant NP-difficile, car le nombre de combinaisons possibles croît de manière exponentielle avec l'augmentation du nombre de modules, d'étudiants, de salles et de créneaux temporels.

Dans un tel contexte, l'utilisation d'approches exactes basées sur des méthodes d'énumération exhaustive ou de programmation linéaire devient rapidement inapplicable pour des jeux de données de grande taille. Afin de répondre aux contraintes de performance du projet, une approche heuristique gloutonne a été retenue.

Cette approche consiste à construire progressivement une solution en effectuant, à chaque étape, un choix local jugé pertinent, sans garantir l'optimalité globale, mais en assurant une solution satisfaisante dans des délais raisonnables. Afin d'améliorer la qualité des solutions générées, cette heuristique est combinée à une stratégie de parcours circulaire des créneaux temporels, connue sous le nom de Round-Robin.

5.2 Justification

Le choix de l'approche heuristique gloutonne est motivé par plusieurs considérations fondamentales.

Tout d'abord, cette approche permet de garantir des temps d'exécution compatibles avec une utilisation réelle du système, même en présence de volumes importants de données, comme plusieurs milliers de modules et des centaines de milliers d'inscriptions étudiantes.

Ensuite, l'heuristique gloutonne offre une grande flexibilité face à l'évolution des contraintes métier. De nouvelles règles académiques ou organisationnelles peuvent être intégrées sans nécessiter une refonte complète de l'algorithme.

Par ailleurs, l'introduction d'un mécanisme de rotation des créneaux temporels permet d'améliorer la répartition des examens sur l'ensemble de la période d'examens. Cette stratégie évite la concentration excessive des examens sur les premières journées et contribue à un meilleur équilibre pour les étudiants et les enseignants.

Enfin, cette approche présente un excellent compromis entre la qualité des solutions produites et la complexité de mise en œuvre, ce qui la rend particulièrement adaptée à un projet académique appliqué.

5.3 Algorithme d'optimisation

Objectif de l'algorithme

L'objectif principal de l'algorithme est de générer un emploi du temps d'examens cohérent en affectant chaque module à un créneau temporel et à une salle appropriée, tout en respectant

l'ensemble des contraintes académiques, humaines et logistiques.

L'algorithme cherche à minimiser les conflits pour les étudiants, à assurer une utilisation efficace des ressources matérielles et à garantir une répartition équitable des charges de surveillance entre les enseignants.

Étape 1 : Préparation et structuration des données

La première étape consiste à préparer l'ensemble des données nécessaires à l'optimisation. Cette phase est cruciale car elle permet de réduire considérablement la complexité des traitements ultérieurs.

Les modules nécessitant un examen sont tout d'abord identifiés et chargés. Pour chacun d'eux, le nombre total d'étudiants inscrits est calculé à partir des données d'inscription. La liste détaillée des étudiants inscrits à chaque module est également récupérée afin de faciliter la détection des conflits.

En parallèle, l'ensemble des salles disponibles est chargé, en tenant compte de leur capacité réelle en période d'examens ainsi que des équipements spécifiques éventuellement requis. Les créneaux temporels disponibles sont également générés à partir de la période d'examens définie.

Enfin, les modules sont triés par ordre décroissant du nombre d'étudiants. Cette stratégie permet de traiter en priorité les examens les plus contraignants, qui sont également les plus difficiles à placer.

Étape 2 : Placement des examens par stratégie Round-Robin

Les modules sont traités successivement selon l'ordre défini à l'étape précédente. Pour chaque module, l'algorithme parcourt les créneaux temporels disponibles selon une stratégie circulaire (Round-Robin). Cela signifie que le point de départ du parcours change à chaque module, évitant ainsi de privilégier systématiquement les mêmes créneaux.

Pour chaque créneau testé, plusieurs contraintes sont vérifiées de manière séquentielle. Tout d'abord, les créneaux correspondant à des journées non autorisées, telles que le vendredi, sont ignorés. Ensuite, l'algorithme vérifie qu'aucun étudiant du module ne passe déjà un examen le même jour.

Une contrainte supplémentaire vise à limiter la charge étudiante en vérifiant qu'aucun étudiant ne possède un examen la veille ou le lendemain du créneau considéré. Cette règle contribue à améliorer le confort académique des étudiants.

Enfin, l'algorithme vérifie la disponibilité du couple salle-créneau ainsi que la capacité de la salle à accueillir l'ensemble des étudiants du module. Si toutes ces contraintes sont satisfaites, l'examen est immédiatement placé.

Lorsque l'examen est placé, la salle et le créneau sont marqués comme occupés et les dates d'examen sont mémorisées pour l'ensemble des étudiants concernés.

Si aucun créneau valide n'est trouvé pour un module donné, celui-ci est temporairement ignoré et n'est pas planifié lors de cette exécution.

Étape 3 : Assignment des surveillants

Une fois les examens positionnés dans le temps et dans l'espace, l'algorithme procède à l'assignation des enseignants chargés de la surveillance.

Pour chaque examen, le nombre de surveillants nécessaires est calculé selon une règle simple : un surveillant est requis pour quarante étudiants. Les enseignants disponibles sont ensuite sélectionnés en tenant compte de leur disponibilité quotidienne et de leur charge globale de surveillance.

Afin de garantir l'équité, l'algorithme limite le nombre maximal de surveillances par jour à trois pour chaque enseignant et privilégie l'affectation des enseignants appartenant au même département que le module concerné.

Étape 4 : Sauvegarde et calcul des indicateurs

À l'issue du processus de génération, l'ensemble des examens planifiés est enregistré dans la base de données. Des indicateurs globaux sont alors calculés, notamment le nombre total d'examens planifiés, le taux d'occupation des salles et la répartition des surveillances entre les enseignants.

Ces indicateurs permettent d'évaluer la qualité du planning généré et servent de base à la validation par les responsables académiques.

Étape 5 : Publication du planning

Une fois validé par l'administrateur, le planning peut être publié. La publication rend le planning accessible aux étudiants et aux enseignants via la plateforme. Toute modification ultérieure nécessite une nouvelle exécution de l'algorithme, garantissant ainsi la cohérence globale des données diffusées.

Synthèse

L'algorithme proposé repose sur une combinaison efficace de tri stratégique, d'heuristique gloutonne et de parcours circulaire des créneaux. Cette méthodologie permet de générer des emplois du temps réalistes, équilibrés et compatibles avec les contraintes opérationnelles d'une université de grande taille.

6 Architecture technique

6.1 Choix des technologies

Dans le cadre de ce projet, une architecture web moderne de type client–serveur a été adoptée, reposant sur une séparation claire entre le frontend, le backend et la base de données. Ce choix permet d'assurer une meilleure maintenabilité, une évolutivité plus simple et une répartition efficace des responsabilités.

Next.js (Frontend)

Le frontend de la plateforme est développé avec **Next.js**, un framework JavaScript basé sur React. Next.js offre un système de routage automatique basé sur la structure des fichiers, ce qui facilite l'organisation des pages et la navigation dans l'application.

Next.js permet également le **Server Side Rendering (SSR)**, c'est-à-dire que certaines pages peuvent être générées côté serveur avant d'être envoyées au navigateur. Cette approche améliore les performances, réduit le temps de chargement initial et optimise le référencement (SEO).

En plus de cela, Next.js propose une architecture de projet claire, une bonne intégration avec les API, ainsi qu'une grande compatibilité avec les plateformes de déploiement modernes, notamment Vercel, ce qui en fait un choix adapté pour une application web.

Express.js (Backend)

Le backend est développé avec **Express.js**, un framework JavaScript léger et largement utilisé pour la création d'API web. Express.js a été choisi afin de rester dans un environnement JavaScript homogène entre le frontend et le backend.

Il permet de gérer efficacement les requêtes HTTP, de structurer les routes de l'application et d'exécuter la logique métier du projet. Dans ce projet, Express.js est principalement responsable de l'exécution de l'algorithme de génération des emplois du temps, ainsi que du traitement des données échangées avec la base de données.

La simplicité d'Express.js, sa rapidité d'exécution et la richesse de son écosystème de bibliothèques sont parfait pour un projet de ce type.

Supabase (Base de données et authentification)

La base de données est gérée via **Supabase**, une plateforme open-source reposant sur PostgreSQL. Supabase offre une solution complète incluant :

- une base de données relationnelle SQL ;
- un système d'authentification intégré ;
- un service de stockage ;
- une API automatique.

Dans ce projet, Supabase est utilisé principalement pour la gestion des données académiques ainsi que pour l'authentification des utilisateurs lors de la connexion. Lors du login, Supabase permet de vérifier l'identité de l'utilisateur et de lui attribuer un rôle spécifique, ce qui conditionne ensuite ses droits d'accès dans la plateforme.

La simplicité d'utilisation, la version gratuite et l'intégration facile avec JavaScript ont motivé le choix de Supabase.

6.2 Structure générale du projet

Le projet est structuré en deux parties principales, chacune disposant de son propre dépôt :

- un dépôt frontend pour l'interface utilisateur ;
- un dépôt backend pour la logique métier et l'algorithme.

Cette séparation permet une meilleure organisation du code, une indépendance entre les équipes de développement et une plus grande flexibilité lors du déploiement.

6.3 Structure du code

Structure du frontend

Le frontend suit l'architecture standard proposée par Next.js. Les principaux dossiers sont :

- **app** : contient les pages principales de l'application, telles que le tableau de bord, la génération du planning et les ressources accessibles via la barre de navigation.
- **src** : regroupe les utilitaires du projet, les appels aux API, les bibliothèques internes et les composants communs comme la barre de navigation.
- **public** : destiné au stockage des images et fichiers statiques.

Structure du backend

Le backend Express est organisé selon les dossiers suivants :

- **middleware** : contient les fonctions d'authentification permettant de vérifier l'identité des utilisateurs et leurs rôles via Supabase.
- **routes** : regroupe les routes de l'API ainsi que les fonctions de traitement associées, incluant la logique de génération du planning.
- **scripts** : contient les scripts permettant de générer automatiquement des données telles que les professeurs, les salles ou les étudiants.
- **migrations** : permet de conserver l'historique des modifications apportées à la base de données.

7 Déploiement et utilisation

7.1 Choix des plateformes de déploiement

Le déploiement du projet repose sur deux plateformes principales :

- **Vercel** pour le frontend Next.js ;
- **Render** pour le backend Express.js.

Vercel est la plateforme officielle développée par l'équipe de Next.js, ce qui garantit une compatibilité optimale, une configuration simplifiée et des performances élevées pour le rendu des pages.

Render a été choisi pour le backend en raison de la qualité de son offre gratuite, de sa stabilité et de sa facilité de configuration par rapport à d'autres alternatives disponibles.

La base de données Supabase étant hébergée sur ses propres serveurs, elle ne nécessite aucune configuration de déploiement supplémentaire.

7.2 Processus d'utilisation

L'utilisateur accède à la plateforme via le navigateur. Après authentification, il peut consulter ou générer les emplois du temps selon son rôle. Lorsqu'un planning est généré, la requête est

transmise au backend qui exécute l'algorithme, enregistre les résultats dans la base de données et renvoie les informations au frontend pour affichage.

8 Conclusion

Ce rapport a présenté la conception et la réalisation d'une plateforme d'optimisation des emplois du temps d'examens universitaires, répondant à une problématique organisationnelle complexe et fortement contrainte. À travers l'analyse des besoins, la modélisation de la base de données, la spécification fonctionnelle et la méthodologie d'optimisation, une solution cohérente et structurée a été proposée.

L'approche retenue repose sur une heuristique gloutonne combinée à une stratégie Round-Robin, permettant de générer des emplois du temps réalistes tout en respectant les principales contraintes académiques, humaines et logistiques. Cette méthode offre un compromis efficace entre qualité des résultats et temps de calcul, ce qui la rend adaptée à un contexte universitaire réel.

L'architecture technique adoptée, basée sur Next.js pour le frontend, Express.js pour le backend et Supabase pour la gestion des données et de l'authentification, assure une séparation claire des responsabilités, une bonne maintenabilité du système et une intégration fluide entre les différentes composantes de la plateforme. Le déploiement sur des plateformes modernes permet également une utilisation simple et accessible via le web.

Ainsi, ce projet met en évidence l'intérêt des solutions informatiques et des méthodes d'optimisation pour améliorer la gestion des processus académiques. Il constitue une démonstration concrète de l'application des concepts d'intelligence artificielle, de bases de données et de développement web à une problématique réelle du milieu universitaire.

Remarque

Afin de faciliter les tests et l'évaluation de la plateforme, les comptes suivants sont mis à disposition :

- **Doyen** : naceryousfi007@gmail.com
- **Administrateur des examens** : mader2000003@gmail.com
- **Étudiant** : elias.bertrand.etd@univ-demo.fr
- **Professeur** : emma.fournier@univ-demo.fr

Le mot de passe est identique pour tous les comptes : **minecraft**

Il est également important de noter que lors du premier accès après connexion, la plateforme peut nécessiter un léger temps de chargement en raison de l'initialisation des services backend. Une fois cette étape passée, l'application devient fluide et rapide lors des utilisations suivantes.